Building AI Systems — Handout

Contents

Generated by the agentic vault — an autonomous research and authoring harness. Every factual claim was gated against a cited source registry before this document was produced.

> ***Reading this document:*** `[S<n>]` *cites the numbered source list at the end.* `[inference]` *marks a claim that follows from those sources but is not stated by them.* `[speculative]` *marks the author's judgement, offered as judgement.*

Ondrej (Ondra) Krajicek · ARTIN · 2026-08-18

Working thesis — my own claim, offered to be argued with, not accepted on faith: *building reliable AI systems is an engineering discipline, not a prompting one; reliability comes from external grounding and bounded autonomy, not from asking the model to check its own work.* [speculative] This handout is the standalone version of the talk, and it goes a layer deeper than the slides. [speculative] Every factual claim traces to a numbered source in the References section below; the three vault research reports behind it carry the deeper citation chains. [speculative]

---

1 1 · Why "prompt engineering" stopped being the interesting problem

For two years the craft was the prompt. That is over — not because prompting is worthless, but because it has been demoted. Prompt engineering "did not die but demoted itself to the innermost layer of context engineering," and its 2025–2026 research frontier is *negative* results: chain-of-thought and expert-persona prompting can actively harm reasoning models, so "always add reasoning steps" is now a per-model empirical question rather than a universal rule [S1].

The layer that wraps the prompt is context engineering, whose central move is to treat "the window as a capped per-turn budget to allocate — not a bucket to fill," and whose maturity is measured by whether context assembly is "a deterministic, testable pipeline" [S2]. The instinct to pour more context into the window backfires: *context rot* means "accuracy degrades as length grows even with irrelevant tokens masked, observed across 18 LLMs, with agentic failure rising ~4× at doubled duration" [S3]. More context is frequently less reliability, and that is a measured result, not an intuition [inference].

The consequence is a change of centre of gravity. The recurring failure modes of long-running agents "are not solved by a smarter model. They are solved by infrastructure" [S24]. Two teams on the same frontier model can ship wildly different reliability; the difference is the system around the model, not the weights inside it [S24]. The interesting question stopped being *what do I type* and became *what do I build* [inference].

---

2 2 · The five pillars

The research frames a reliable AI system as five pillars: **prompt** (the innermost layer) [S1], **context** (a budget to allocate) [S2], **grounding** (condition on evidence, then verify) [S4], **orchestration** (compose agents, splitting maker from checker) [S26], and **bounded autonomy** (freedom inside a fence) [S7]. They are not five topics but one spine, and the spine is clearest in the grounding pillar [speculative].

Grounding decomposes into three epistemically distinct layers — "parametric correlation, structural grounding, and deterministic verification — and reliability comes not from a more capable model but from closing a generation–verification loop against an external oracle where the LLM is orchestrator, not load-bearer" [S4]. An *oracle*, here, is simply a check that can tell right from wrong without asking the model — a test's exit code, a type checker, a solver's verdict, a schema validator — its defining property being negative: the generator cannot satisfy it by *asserting* it is done [inference]. Only the deterministic layer gives a genuine causal guarantee; structural (retrieval) grounding improves the odds without certifying anything — "only deterministic verification — where the LLM is not the load-bearing component — approaches a genuine causal guarantee (Layer 3)" [S41]. The distinction is not academic — it decides where the verifier sits in the architecture: an advisory answer can ride on retrieval, but an irreversible action needs a deterministic check standing between the model and the effect [inference].

The headline result makes it concrete, and I offer it as external evidence — it comes from a safety-critical engineering-compliance domain that is not this audience's own, cited only because it is the cleanest external-verifier result available. In a closed-loop 2026 study, adding an external FE (finite-element) verifier moved the *same agents* from 56.8% to 98.6% compliance, and the gain held across two model families, DeepSeek and Claude [S5]. Nothing about the model changed; the generation–verification loop did — and because "the guarantee now lives in the verifier, not the generator," swapping the generator does not move it [S15]. The mechanism generalises, but not without bound, and the report flags the study's own boundary condition honestly — "verification loops fix the errors the verifier can see, and no more": when a defect originated in structural topology rather than the parameters the FE verifier checks, the parameter-level closed loop was, in the authors' word, "powerless," so the guarantee transfers across domains only as far as the scope of the oracle you actually build [S37]. That is the general pattern: "reliability is bought by *relocating the truth-determining computation out of the probabilistic model*" [S6].

Orchestration is where the loop discipline of section 3 first appears as an architecture: the load-bearing move is splitting the agent that does the work — the *maker* — from the agent that checks it — the *checker* — giving the checker an isolated context so it cannot simply confirm the maker's reasoning [S26]. I use *maker* and *checker* throughout for these two roles; the research papers name the same split *generator* versus *verifier* [S15] or talk about the independence of the *observer* [S9] — one split under several names. The same discipline plausibly extends to the *shape* of a multi-agent system — the interface between agents is the design, not the head-count [inference] — and a single production rebuild is one data point for that reading, not proof of a field-wide law. Intuit rebuilt its agent architecture twice in about four months, and it is worth being precise that two rebuilds is two decisions with only one stated cause. The *second* re-

build abandoned natural-language agent-to-agent handoffs because "a ten agent chain did not fail occasionally, it compounded errors by design," settling on "skills-and-tools, not agent-to-agent natural-language delegation" [S27]. The *first* — a specialist-agent fleet becoming a central orchestration layer — has no cause in the source, which reports the endpoint rather than that step, so I decline to invent one [inference]. A free-text handoff is an untyped interface between unreliable components; the fix is a typed one [inference]. The fifth pillar is governance: "unbounded autonomy in complex systems inherently creates unbounded risk" [S7], so autonomy is a calibrated design parameter, not a default, with humans retaining authority over the boundary conditions [S43]. The counter-intuitive part — the one worth pausing on — is that a good bound does not cost capability. In the Physics-in-the-Loop study (a CAD/structural-design domain that is again not this audience's own, cited only for the mechanism), putting a deterministic finite-element oracle inside the agent's decision loop as the fence moved target-safety-factor success from 22.2% to 59.0% ($p = 0.0008$) *and* let the agent produce designs 4.2× more structurally complex — "FEA-enabled runs hit the target safety factor 59.0% of the time versus 22.2% with the physics tool disabled" [S45]. The deterministic bound is not a leash that shortens the agent's reach; it is the floor that lets it explore aggressively without falling through — freedom inside a fence, where the fence is what makes the freedom safe to grant [inference].

---

3 3 · Loops: why self-verification underperforms

There are two shapes of loop: one that closes against an external oracle — a test, a solver, an independent judge — and one that re-invokes the same model to check itself [S9]. The finding that reorganises the field is that "what carries the effect is the independence of the observer, not just the presence of a loop" [S9]. Iteration count is not the load-bearing variable; verifier independence is. Put sharply: "a loop's verdict is only worth what its verifier's independence is worth; buy that independence in the check, or don't call the loop verified" [S10]. One controlled study makes the gap concrete on a single task — auditing an agent's work for real violations. Asked to audit its own output, the agent flagged 0 of 34 real violations at 90–100 confidence; a *fresh* instance of the same model caught 7; a deterministic checker caught all 34 — "a fresh instance of the same model caught 7, and a deterministic checker caught all 34" [S47]. The underlying model is identical across the first two; what changed is how independent the check's evidence is, which is the whole point. The study reports those counts, not what the fresh instance was shown: a fresh instance is independent of the maker only in the weak, context-isolation sense — same weights, a new context — which the experiment does not itself describe but which is the general independence principle at work [inference].

Why can a model not simply grade itself? The report's memorable label is that "the AI self-check wall is Gödel with a leaderboard" [S11]. The step being borrowed, stated plainly so the name is not doing the work: "Gödel showed a formal system rich enough to describe arithmetic cannot prove its own consistency from inside" itself [S46] — borrowed strictly as an *analogy*, not a theorem about neural net-

works; the only honest transfer is that a checker built from the same system as the thing it checks has no vantage point outside it [inference]. The report is deliberately careful not to inflate the metaphor: it rejects the flat slogan "self-checking is impossible" in favour of the narrower, defensible claim that "trustworthy knowledge always has to pay for its own verification; the price is never zero" [S11]. The mechanism underneath is a conditioning one — a self-monitor grades from the same evidence that produced the answer, so re-invoking the same model opens no independent information channel; put another way, an LLM asked to *verify* does not run a verifier, it re-runs the same lossy process, and the fix is a different channel you pay for rather than a smarter grader [inference]. The empirical case points the same way, though the sharpest number is scoped and should be quoted as such. A single 2026 preprint (Khullar et al., arXiv:2603.04582) measured *on-policy self-monitoring* — a model grading output from its own policy, i.e. checking a patch it just produced — across four coding and tool-use datasets. When the model grades a patch it just generated, its discrimination degrades: the monitor's AUROC (the *area under the ROC curve* — a right-versus-wrong separation score in which 1.0 is perfect discrimination and 0.5 is chance) falls from 0.99 to 0.89 in the code-correctness setting [S12]. Read against that scale, 0.99 is near-perfect and the drop to 0.89 is a small but real loss a reader can check rather than take on trust [inference], and it appears only because the work is the model's own [S12]. And "in one setting" the study found self-attribution bias "makes it 5 times more likely that a monitor approves a code patch that followed a prompt injection" [S12]. Because this is a single preprint, the report's own Skeptic reading applies: these magnitudes are "directionally robust but not settled," so the load-bearing claim is the *direction* — a self-monitor is weakest exactly where authorship bias bites, i.e. under attack — not the precise 5× multiple [S12]. And on formal verification, "rule-based approaches remain more reliable for verification success, while LLM-based methods exhibit more variable performance" [S13]. The paper's own framing is the careful one — not "rules beat LLMs" but rules *more reliable*, LLMs *more variable and complementary* — so the practitioner reading is a priority ordering: default the check to a deterministic gate, and reach for a model judge only where the target is irreducibly subjective [inference].

The operational rule follows directly: "never let a loop close against the same model that opened it; close it against an external oracle, or don't call it verification" [S14]. In practice the checker gets an isolated context — it sees the task and the result, never the maker's reasoning [S26] — and two circuit breakers, an iteration ceiling and a token budget, keep an unattended loop safe [S26]. Add the checker before you add the autonomy [speculative].

---

4 4 · The harness — the system around the model

The harness is everything in an agent except the model: tools, sandboxes, memory, sensors, permissions, orchestration. Its payback is now measured. OpenAI's team shipped "roughly **1,000,000 lines of production code via ~1,500 automated PRs with zero manually written code**," and LangChain lifted a coding agent "**52.8 → 66.5** … *purely by harness changes, with the model held constant*" [S25], both first-party accounts [S34][S35]. At the frontier, harness work pays back faster than a model swap [speculative].

Running through every harness layer — permissions, memory, evaluation, orchestration, guardrails [S24] — is one law: "push determinism to the load-bearing control, and bound the probabilistic model inside it" [S16]. Permissions are the worked example. Progent, a deterministic runtime privilege control verified with an **SMT** (Satisfiability Modulo Theories) solver, "provably reduces attack success rate to 0% with hand-written policies, while prompt-level defenses are demonstrably bypassed" [S17], benchmarked independently on AgentDojo and **ASB** (Agent Security Bench) [S32]. It is worth separating what "provable" attaches to, because the word does two jobs and only one is a proof: the SMT verification proves the *mechanism* — the policy engine provably enforces whatever policy it is handed, a property the paper calls monotonic confinement [S17] — while the *0% attack-success figure* is not a theorem but a *measured* result, obtained by evaluating that mechanism on the AgentDojo and ASB benchmarks under hand-written policies [S32]. Proven mechanism, measured result: not the same claim, and a precise room will hold you to the difference.

The nuance matters and must not be flattened to "near-zero." That "provable 0% attack-success-rate is a *manual-policy* result; the fully-autonomous LLM-generated-policy configuration leaves residual ASR (AgentDojo 41.2%→2.2%, ASB 70.3%→7.3%)" [S18]. Even deterministic control degrades once the model writes its own policy — so the human owns the policy and the machine executes it [inference]. A privilege policy attaches to a tool-class, not a task, so the hand-written slice amortises across every task that touches that tool [inference]. For reference, the primary Progent paper's headline configuration reports AgentDojo 39.9%→1.0% and ASB 70.3%→3.9%; the exact figure depends on who authors the policy and which evaluation, which is precisely the point [S32].

The harness is also the attack surface. "Agentjacking (poisoned MCP tools, fake bug reports; 100+ coding agents compromised) … establish that 'prompt-layer defenses fail; execution-layer controls are required'" [S19]. Cite the corroborated number — "over 100 AI coding agents compromised across Fortune 500 and Fortune 100 companies" [S29] — and *not* the headline 85% success rate, which "remains single-source (Tenet Security) as of July 2026" and has not been independently reproduced [S30]. That surface is still widening rather than narrowing: GhostJacking, disclosed in August 2026, extends the same technique by poisoning trusted logs and alerts to make agents run attacker code [S36].

Two honest caveats keep this from becoming cargo cult. First, not every harness change helps: "Codex dropped from 93.1% to 55.2% when switching from inline to file-based grep delivery" — one tool-delivery change, roughly 38 points, same model [S28]. The specific evidence is worth stating rather than the conclusion alone: the only thing that changed was how tool output reached the model — inline text

versus written to a file — and the authors read file-based routing as trading context bandwidth for compositional tool competence, where "gains are realized only when the agent reliably closes the loop" [S28]. Whether Codex specifically failed to close that loop, the paper does not say: it reports the collapse without error analysis for why particular harnesses fail at file-based workflows, so on the note's own reading the finding is "descriptive rather than explanatory" [S28]. The honest takeaway is therefore the *direction* — a cosmetic-looking delivery choice can swing accuracy by forty points — not a validated causal mechanism [inference]. The harness is leverage in both directions, which is why you measure changes against an eval set [inference]. Second, harnesses decay: a scaffold built for a model's weakness becomes overhead — and can cap capability — once that weakness disappears, so harnesses need deletion on a cadence, not just accretion [S44]. The cadence the source actually supports is *model-driven*, not calendar-driven — a "periodic entropy audit" run whenever the model changes: "re-run the harness against your eval set on every model bump" [S44]. Its body text states no fixed calendar interval — it names no "quarterly" or other number, so I supply none [inference] — and it concerns the human-authored scaffolding teams accumulate; the source makes no claim about AI-generated harnesses, so neither will I [inference].

The most concrete anchor is a production incident this audience will recognise on sight. Amazon's Kiro agent "autonomously decided to delete and rebuild the production environment" — AWS Cost Explorer, one region, a roughly 13-hour outage — because it had authority over an irreversible action and no deterministic gate stood between the decision and the effect [S20]. The response is the tell: not a smarter agent, but "an SVP-driven 90-day safety reset touching ~335 of its most important systems" and retrofitted mandatory two-person sign-off and senior-engineer approval for AI-assisted production changes [S20]. That is bounded autonomy added after the incident instead of before it — a permission-scoping failure of exactly the kind delivery teams already guard against for humans [inference]. Nor is this a Big-Tech-only story: Salesforce runs "guided determinism," an agent graph in which "some transitions are guarded by hard validation gates that have to pass before execution can continue" [S33] — a different industry, the same law.

---

5 5 · What this means for the room on Monday

As generation gets cheap, "the binding constraint migrates downstream" — off producing the artifact and onto understanding, validating, integrating and owning it [S21]. Spotify's own engineering organisation reports "76% more PRs to review" once agents ramped [S31]. The trade to watch: cheap generation moves the constraint to review, it does not remove it, and quality debt surfaces downstream if review capacity cannot keep pace [S21]. The payoff of harness engineering is therefore not fewer humans — it is redirecting human effort to review and verification [inference].

So the job shifts. "The engineer's value is defined not by what the agent generates but by the control plane the engineer builds to constrain, verify, and absorb it" [S23]. That moves you up the stack, not out of it [speculative]. The recipe is deterministic-first: "make the deterministic component load-bearing, make the model the orchestrator around it, and bound the whole with calibrated human oversight" [S8], and close every loop against an external oracle [S14].

One honest limit belongs here, because a hostile expert will raise it. The strongest evidence for external grounding comes from domains that happen to own a cheap, trustworthy, independent oracle — a finite-element solver, a test suite, a schema check. And even where such an oracle exists, its scope bounds the guarantee: a verification loop fixes only the errors the verifier can see, which is why the flagship 56.8→98.6 study still leaves defects of structural topology — outside its parameter-level FE verifier — uncaught, a limit its own authors flag [S37]. So before adopting, triage the task into one of three zones, each recognisable from your own work: *oracle-rich* (a schema validator on an API response, a contract test on a service boundary, a migration dry-run — close the loop and get the guarantee) [inference]; *oracle-scoped* (a type checker proves your types are sound but says nothing about whether the business logic is right — the loop certifies only what it checks) [S37]; and *oracle-free* (is this refactor correct, is this summary good — no complete oracle at all) [inference]. Most open-ended production work sits in that last zone, and there external grounding does not save you outright — but "no complete oracle" rarely means "no check." A senior team already owns cheaper, *partial* oracles that raise the floor without certifying, and each maps onto something you already run: property-based tests assert invariants rather than exact outputs — the workhorse being a round-trip, assert that deserialize(serialize(x)) equals x for random x — and, "with the best model and prompting approach," a valid, sound one "can be synthesized in 2.4 samples on average," though that same best model synthesizes correct tests for only 21% of the properties extractable from API documentation, so the figure is a best-case result, not a general rate [S38]; metamorphic testing checks that equivalent inputs give consistent outputs with no ground truth — reorder two independent query filters, or rename a local variable, and the result must not change — detecting "75% of the erroneous programs generated by GPT-4, with a false positive rate of 8.6%" [S39]; canary and shadow deployment make production itself the oracle over "a small portion of your user traffic" — route 1% to the new agent and alarm on the error-rate delta — at a bounded blast radius [S40]; and idempotency and reversibility make a wrong action safe to retry rather than catastrophic — an idempotent write behind a key, or a migration with a real down-step — so "executing an operation once has the same effect as executing it multiple times" [S42]. None of these is as clean as a finite-element solver, but stacked they usually turn "no oracle" into "good-enough oracle" [speculative]. Where even that is impossible, the honest architecture is bounded autonomy plus human review, and sometimes the right engineering call is to refuse to automate the decision at all. Knowing which zone you are standing in — oracle-rich, oracle-scoped, or oracle-free — is itself part of the job [speculative].

A caveat on vocabulary: the labels — harness, loop, graph engineering — are contested, so "advocate the engineering idea; hedge the compound noun" [S26]. The durable idea survives whatever the term of art becomes [speculative].

Three moves for Monday. (1) Find one self-graded loop and give it an *independent* check [S14]. (2) Put one *deterministic gate* on an irreversible action — a merge, a deploy, a write [S8]. (3) Take one figure you repeat about AI and confirm it is corroborated, not single-source [S30]. None of these needs a better model; all of them are engineering [inference].

What follows from the thesis. Reliability is bought by relocating truth-determination out of the model [S6]; independence — not iteration — is the load-bearing variable [S10]; push determinism to the control and bound the model inside it [S16]; and autonomy is the property you add *last* [inference]. Or, in one line: "the model is a commodity; the harness is the product" [S22]. Reliability is engineered, not prompted [speculative].

---

6 References

Full registry, including the exact vault spans relied upon and per-web-source credibility notes, is in `sources.md`. Vault sources are notes in this repository; web sources carry a URL.

1. [S1] Building AI Systems: 5 Pillars — comprehensive research report (vault). Prompt engineering demoted to the innermost layer of context engineering.
2. [S2] 5 Pillars — context engineering (vault). Context as a capped budget; assembly as a deterministic, testable pipeline.
3. [S3] 5 Pillars — context rot (vault). Accuracy degrades with length across 18 LLMs; agentic failure ~4× at doubled duration.
4. [S4] 5 Pillars — grounding in three layers (vault). Parametric correlation, structural grounding, deterministic verification.
5. [S5] 5 Pillars — closed-loop external-verifier result (vault). Same agents 56.8% → 98.6% compliance via an external FE (finite-element) verifier, model-invariant across DeepSeek and Claude; cited as external evidence from a safety-critical domain that is not the audience's own.
6. [S6] 5 Pillars — relocating truth-determination (vault). Reliability is bought by moving truth-determination out of the model.
7. [S7] 5 Pillars — bounded autonomy (vault). Unbounded autonomy in complex systems creates unbounded risk.
8. [S8] 5 Pillars — practitioner synthesis (vault). Deterministic component load-bearing, model as orchestrator, calibrated human oversight.
9. [S9] Loop Engineering — the two loop archetypes (vault). Independence of the observer carries the effect, not the loop.
10. [S10] Loop Engineering — verifier independence (vault). A loop's verdict is worth only its verifier's independence.

11. [S11] Loop Engineering — comprehensive report (vault). The "Gödel with a leaderboard" framing, honestly narrowed: not "self-checking is impossible," but "trustworthy knowledge always has to pay for its own verification; the price is never zero."
12. [S12] Loop Engineering — the empirical case (vault). Scoped result from a single 2026 preprint (Khullar et al., arXiv:2603.04582): on-policy self-monitoring degrades discrimination across four coding/tool-use datasets (AUROC 0.99→0.89, code-correctness setting), and "in one setting" self-attribution bias makes a monitor 5× likelier to approve a prompt-injected patch — read as directionally robust, not settled.
13. [S13] Loop Engineering — the empirical case (vault). Rule-based verification beats variable LLM self-critique.
14. [S14] Loop Engineering — practitioner synthesis (vault). Never close a loop against the model that opened it.
15. [S15] Loop Engineering — practitioner synthesis (vault). The guarantee lives in the verifier, not the generator.
16. [S16] Harness Engineering — comprehensive research report (vault). Push determinism to the load-bearing control; bound the model inside it.
17. [S17] Harness Engineering (vault). Progent reduces attack success to 0% with hand-written policies; prompt defenses bypassed.
18. [S18] Harness Engineering (vault). LLM-generated Progent policies leave residual ASR (AgentDojo 41.2%→2.2%, ASB 70.3%→7.3%).
19. [S19] Harness Engineering (vault). Agentjacking; 100+ agents compromised; execution-layer controls required.
20. [S20] Coding Agent Horror Stories: The Agent That Deleted Production — Docker (2026), corroborated in a vault watch note. Amazon Kiro deleted production (AWS Cost Explorer, ~13-hour outage); SVP-driven 90-day safety reset across ~335 systems; two-person sign-off and senior-engineer approval retrofitted for AI-assisted production changes. https://www.docker.com/blog/coding-agent-horror-stories-the-agent-that-deleted-production/
21. [S21] Harness Engineering (vault). The binding constraint migrates downstream as generation gets cheap.
22. [S22] Harness Engineering — practitioner synthesis (vault). The model is a commodity; the harness is the product.
23. [S23] Harness Engineering — practitioner synthesis (vault). Value is the control plane the engineer builds to constrain, verify, absorb.
24. [S24] Harness Engineering (vault concept note). Long-running-agent failures are solved by infrastructure, not a smarter model.
25. [S25] Harness Engineering (vault concept note). OpenAI ~1M lines via ~1,500 automated PRs; LangChain +13.7 pts, model held constant.

26. [S26] Loop Engineering (vault concept note). Advocate the engineering idea; hedge the compound noun.
27. [S27] Intuit Scrapped Its AI Agent Architecture Twice in Four Months — VentureBeat, VB Transform 2026, corroborated in a vault watch note. Natural-language agent handoffs "compounded errors by design"; final architecture skills-and-tools, not agent-to-agent delegation. https://venturebeat.com/orchestration/intuit-scrapped-its-own-ai-agent-architecture-twice-in-four-months-at-vb-transform-2026-its-ai-vp-called-that-the-fast-path
28. [S28] Is Grep All You Need? How Agent Harnesses Reshape Agentic Search (vault reading note). Codex 93.1% → 55.2% from one tool-delivery change.
29. [S29] Over 100 AI Coding Agents Taken Over Via Agentjacking (vault watch note). 100+ agents across Fortune 500/100.
30. [S30] 85% Agentjacking Success Rate Not Independently Replicated (vault watch note). The 85% figure remains single-source (Tenet Security).
31. [S31] Coding Is No Longer the Constraint — Spotify Engineering. 76% more PRs to review once agents ramped. https://engineering.atspotify.com/2026/6/code-with-claude-coding-is-no-longer-the-constraint
32. [S32] Progent: Securing AI Agents with Privilege Control — arXiv 2504.11703. SMT-verified privilege control; headline AgentDojo 39.9%→1.0%, ASB 70.3%→3.9%. https://arxiv.org/abs/2504.11703
33. [S33] Building Enterprise AI Agents That Are Both Autonomous and Reliable — Salesforce Engineering. "Guided determinism": hard validation gates in an agent graph. https://engineering.salesforce.com/building-enterprise-ai-agents-that-are-both-autonomous-and-reliable/
34. [S34] Harness Engineering: Leveraging Codex in an Agent-First World — OpenAI. ~1M lines / ~1,500 PRs, zero hand-written code. https://openai.com/index/harness-engineering/
35. [S35] Improving Deep Agents with Harness Engineering — LangChain. Terminal Bench 52.8 → 66.5 from harness changes, model held constant. https://blog.langchain.com/improving-deep-agents-with-harness-engineering/
36. [S36] ThreatsDay: GhostJacking AI Attacks — The Hacker News (August 2026). GhostJacking extends agentjacking via poisoned logs/alerts. https://thehackernews.com/2026/08/threatsday-ghostjacking-ai-attacks.html
37. [S37] 5 Pillars — Grounding (Aspect 3), the closed-loop boundary condition (vault). A verification loop fixes only the errors its verifier can see; the flagship 56.8→98.6 result leaves structural-topology defects — outside its parameter-level FE verifier — uncaught, so the guarantee is bounded by the verifier's scope, not the domain. The study's authors flag this limit themselves.

38. [S38] Property-Based Testing for LLMs — arXiv:2307.04346, via the vault note *Alternative Methods to LLM Evaluations*. With the best model and prompting approach, a valid, sound property-based test can be synthesized in ~2.4 samples on average — a best-case figure, since that same best model covers only 21% of properties extractable from API documentation; a cheap partial oracle for oracle-free work. https://arxiv.org/abs/2307.04346
39. [S39] Metamorphic Prompt Testing for LLMs — arXiv:2406.06864, via the vault note *Alternative Methods to LLM Evaluations*. "Overcomes the oracle problem" via consistency checks with no ground truth; detects 75% of erroneous GPT-4 programs at an 8.6% false-positive rate. https://arxiv.org/html/2406.06864v1
40. [S40] Canary & Shadow Deployment for LLM Apps — vault note *Alternative Methods to LLM Evaluations*. Route a small portion of user traffic to new behaviour; production becomes the oracle at bounded blast radius.
41. [S41] Structural Grounding — vault concept note. Three-layer grounding taxonomy; only deterministic verification, where the LLM is not the load-bearing component, approaches a genuine causal guarantee (Layer 3).
42. [S42] Idempotency in Distributed Systems — vault concept note. An idempotent operation executed once has the same effect as executing it multiple times, so a failed call is safe to retry — a cheap partial oracle.
43. [S43] Bounded Autonomy in AI — vault concept note. Autonomy as a calibrated design parameter with humans keeping authority over the boundary conditions, not a binary property.
44. [S44] Harness Engineering — deletion cadence — vault concept note. Re-run the harness against the eval set on every model bump — a model-driven "periodic entropy audit," with no fixed calendar interval stated in the source's body text; concerns human-authored scaffolding, not AI-generated harnesses.
45. [S45] Physics-in-the-Loop: A Hybrid Agentic Architecture for Validated CAD Engineering Design — Berger et al., IJCAI-ECAI 2026 (arXiv:2605.19717), via the 5 Pillars report. FEA-in-the-loop agents hit the target safety factor 59.0% vs 22.2% with the physics tool disabled ($p = 0.0008$), with a 4.2× increase in structural complexity; cited as external CAD-domain evidence, not the audience's own. https://arxiv.org/abs/2605.19717
46. [S46] Loop Engineering — the theoretical case against self-verification (vault). Gödel's second incompleteness theorem stated plainly ("a formal system rich enough to describe arithmetic cannot prove its own consistency from inside"), borrowed only as an analogy for why a self-monitor cannot certify itself.
47. [S47] Loop Engineering — the two loop archetypes (vault). Controlled self-audit study: the agent auditing its own work caught 0 of 34 real violations, a fresh instance of the same model caught 7, a deterministic checker caught all 34.